

# Línea de producción con balanceo dinámico de carga

Sistema distribuido que simula una planta conservera, detecta su cuello de botella en vivo y demuestra qué pasa cuando se escala la etapa lenta

**César Olivares · Simón García-Huidobro**

Profesor: Daniel Calderón R. · 24 de agosto de 2026

# 1 EL CASO

Una planta conservera ficticia de jurel en lata, con cuatro etapas en serie



- La unidad de trabajo es el **lote**; entre etapa y etapa hay una **cola**: la fila de lotes que esperan.
- Si a una etapa le llega trabajo más rápido de lo que procesa, su fila crece: eso es un **cuello de botella**.
- **Envasado**, la etapa más lenta, se replica detrás de un mecanismo de balanceo.

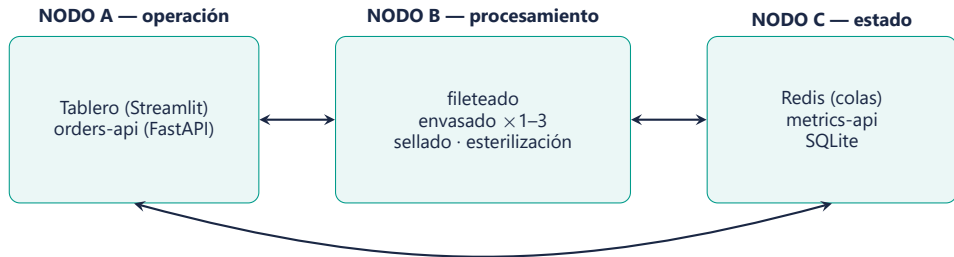
## La pregunta del caso

¿Qué pasa con el cuello de botella cuando escalas la etapa lenta?

**Adelanto: no desaparece — se muda.**

# 2 ARQUITECTURA

Cinco servicios en tres nodos lógicos; cada nodo escala por un motivo distinto



- **Nodo B** es el único que se replica bajo carga: más capacidad = más contenedores ahí.
- **Nodo C** concentra el estado compartido, que no puede duplicarse sin coordinación.
- **Nodo A** escala con los usuarios, no con la producción. Separar nodos separa dominios de falla: si las estaciones saturan la CPU, el operador sigue atendido.

# 3

## UNA IMAGEN, CUATRO ESTACIONES

El mismo programa, desplegado cuatro veces con configuración distinta

- Cada estación es **el mismo código**: la etapa, sus colas y su tiempo de ciclo llegan por variables de entorno (NOMBRE\_ETAPA, COLA\_ENTRADA, COLA\_SALIDA, TIEMPO\_CICLO).
- Ninguna réplica sabe cuántas hermanas tiene, ni necesita saberlo.
- Escalar la etapa replicada es **un solo comando**, sin tocar configuración.

```
docker compose up -d  
--scale envasado=3
```

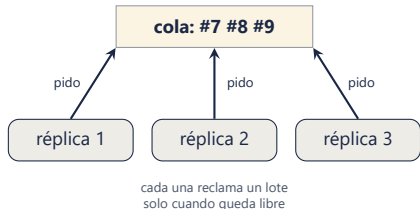
# 1 imagen

4 etapas · hasta 6 contenedores de estación

## 4 BALANCEO POR DEMANDA

Las réplicas piden trabajo al quedar libres; nadie se los asigna

- El enunciado sugiere “réplicas detrás de un balanceador”. Nuestro balanceador **es la cola compartida**: el punto único por el que pasa el trabajo.
- Un round-robin es *estático*: le sigue enviando lotes a una réplica lenta o atascada.
- Con reparto por demanda, la réplica lenta simplemente **pide menos veces**: el balanceo dinámico emerge del mecanismo.



## 5

# CONDICIÓN DE CARRERA

La provocamos antes de resolverla — ambas versiones están en el historial de commits

**Ingenua: mirar y sacar (2 pasos)**



**duplicados con 3 réplicas y 200 órdenes**

**Atómica: BRPOP (1 operación)**



**0 duplicados en 200 órdenes**

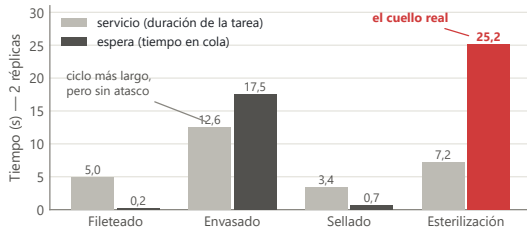
- Entre "mirar" (LRANGE) y "sacar" (LREM) hay una ventana en la que otra réplica ve la misma orden. La ensanchamos a propósito hasta reproducir duplicados.
- BRPOP extrae en **una operación indivisible**: la ventana no se administra con locks y TTL — **se elimina**.

## 6

## CRITERIO DE CUELLO DE BOTELLA

Medimos espera, no servicio — y esa decisión es el corazón del proyecto

- En el supermercado: un cajero lento no es atasco; atasco es la **fila que crece** frente a su caja.
- Cuello = etapa con **mayor espera promedio** en una ventana móvil de 60 s, sobre umbrales relativos a su ciclo (advertencia  $\geq 1,5\times$ , crítico  $\geq 3\times$ ).
- Con 2 réplicas, envasado sigue teniendo el ciclo más largo (12 s) — pero la fila crece en **esterilización**.



# 7

## DEMO EN VIVO

El sistema completo, corriendo — tablero en localhost:8501

---

1. Línea al día: 4 etapas, 5 réplicas visibles con su ciclo y su carga.
2. Ingresamos lotes desde la interfaz, espaciados (~5 s).
3. La espera de esterilización crece en el gráfico: cartel **ámbar** → **rojo**, con la razón del diagnóstico.
4. Matamos Redis: la interfaz **nombra al servicio caído** y el resto sigue vivo. Lo levantamos y todo continúa.

### Qué mirar durante la demo

El color marca *importancia*, no identidad: lo normal es gris; el color fuerte queda reservado para advertencia y crítico, siempre con icono y texto.

# 1 comando

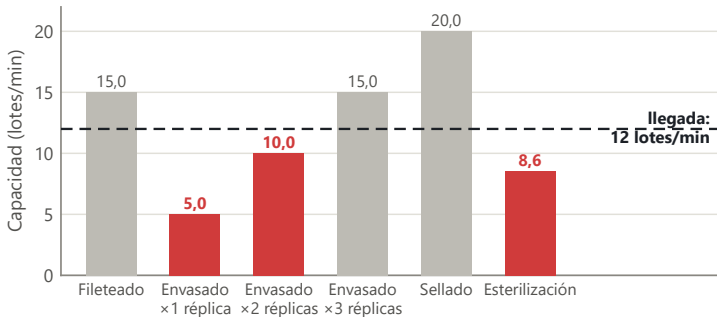
`docker compose up` — despliegue completo



## 8

# EXPERIMENTOS: LA ARITMÉTICA DEL ESCENARIO

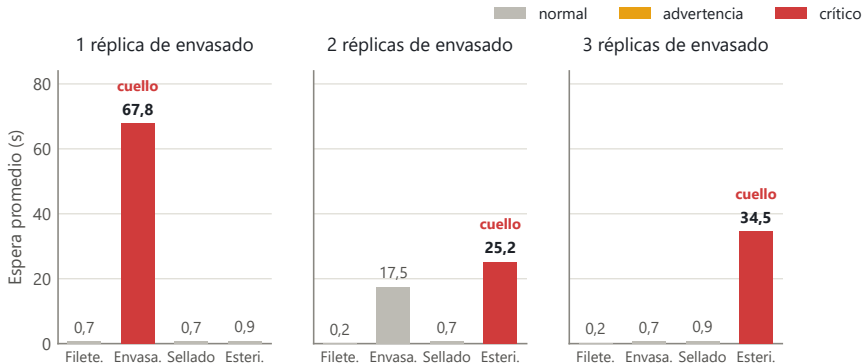
Inyección de 1 orden cada 5 s durante 180 s = 12 lotes/min



- Toda etapa **bajo la línea de llegada** acumula fila tarde o temprano.
- Con 2 réplicas, envasado sube a 10 lotes/min — mejor, pero aún insuficiente — y esterilización (8,6) queda como siguiente límite: la figura **anticipa los resultados**.

# EL RESULTADO CENTRAL

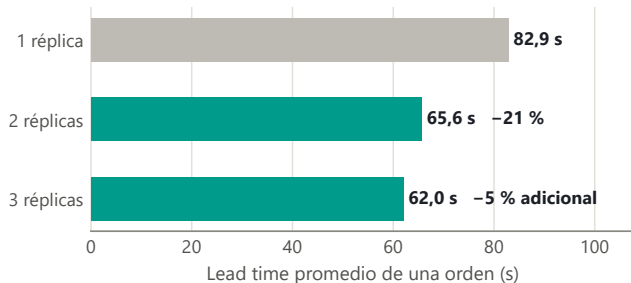
La espera por etapa según réplicas de envasado — el cuello no se elimina: se muda



- Con 1 réplica, envasado acumula 67,8 s de espera; el resto de la línea, casi ociosa.
- Con 2, el cuello **se desplaza a esterilización**; con 3, envasado queda sobrado (0,7 s)... y el cuello sigue en esterilización.

## ¿CUÁNTO COMPRA CADA RÉPLICA?

Lead time: lo que tarda una orden desde que se crea hasta que sale terminada

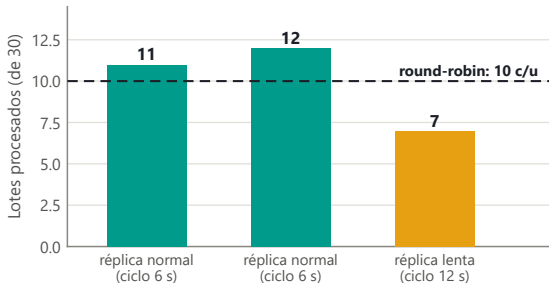


- La segunda réplica compra un **21 %** de mejora; la tercera, casi nada: el límite ya no está en envasado.
- La siguiente inversión correcta sería **una segunda autoclave**, no una tercera envasadora — y el tablero lo responde en vivo.

## 11

## BALANCEO CON UNA RÉPLICA LENTA

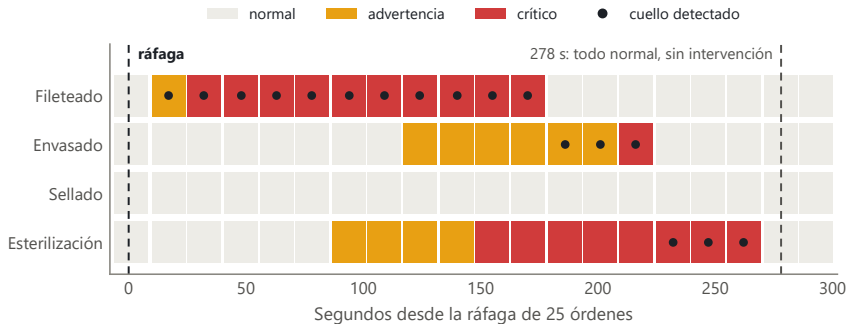
Dos réplicas normales + una al doble de ciclo, sobre la misma cola, 30 lotes



- Un round-robin habría repartido **10/10/10** sin importar la velocidad.
- El reparto **11/12/7** no lo programó nadie: emergió de que cada réplica pide trabajo solo al quedar libre.
- Eso es balanceo *dinámico*: proporcional a la capacidad real de cada una.

# SATURACIÓN Y RECUPERACIÓN AUTOMÁTICA

Ráfaga de 25 órdenes de golpe, sonda a /metricas/cuello cada 15 s



- A los 32 s el sistema declara **crítico** solo; la congestión recorre la línea como una ola.
- A los **278 s** todo vuelve a normal **sin intervención**: las esperas viejas salieron de la ventana móvil.

## MANEJO DE ERRORES

La interfaz nunca muestra un traceback: nombra al servicio que realmente falta

---

46 s

colgado con Redis caído  
(el cuelgue estaba en el DNS, no  
en el socket)

2,1 s

respuesta 503 con plazo duro de  
2 s y mensaje en castellano

- 201 creación · 422 validación · 404 inexistente · 503 dependencia caída — verificados contra el sistema corriendo.
- El healthcheck mide *vida*, no dependencias: un servicio que responde 503 explicándose está sano.

# 14

## PERSISTENCIA Y REPRODUCIBILIDAD

Dos tablas, nada guardado dos veces; despliegue con un comando

- SQLite en un volumen: ordenes y eventos (12 eventos por orden completa).
- Espera, servicio y lead time **no se almacenan**: se calculan siempre desde los eventos — el dato guardado dos veces es el dato que se contradice.
- La base sobrevive a `docker compose down` (verificado).

```
git clone <repo> && cd <dir>  
docker compose up -d --build  
# tablero en localhost:8501
```

# 8/8

contenedores *healthy* en la prueba de clon limpio

# 15

## RESILIENCIA: CÓMO SE ABORDARÍA

No exigida por el enunciado — pero la arquitectura deja los puntos de corte listos

---

- **Estaciones:** sin estado propio, reconectan y mueren sin corromper nada. Pendiente: re-encolar el lote en curso si una réplica muere a mitad de ciclo (BLMOVE a una cola “en proceso” + barrendero de latidos vencidos).
- **Redis:** punto único de falla del reparto → Redis Sentinel, o un broker con confirmaciones (el bonus Kafka apuntaba ahí).
- **SQLite:** con más escritores o failover, migrar a MySQL/PostgreSQL cambiando solo el módulo `bd.py`.
- **APIs y UI:** sin estado; dos instancias tras un balanceador HTTP quedan en alta disponibilidad — los healthchecks ya existentes son su insumo.

Cada mejora toca **un** nodo sin rediseñar los demás: para eso era la división en nodos.



# CONCLUSIONES

Todo lo afirmado está respaldado por mediciones reproducibles

---

## El cuello no se elimina: se muda

De envasado a esterilización al pasar de 1 a 2 réplicas; la tercera ya casi no compra nada.

## Medir espera es lo que permite verlo

El cuello real tenía un ciclo más corto que envasado; por servicio jamás se habría encontrado.

## El balanceo dinámico emerge

11/12/7 con una réplica lenta, sin que nadie asignara ese reparto.

## La carrera se elimina, no se administra

Extracción atómica: 0 duplicados en 200 órdenes, contra los duplicados de la versión ingenua.

## Fallas explicadas, no enmascaradas

503 en 2,1 s nombrando al culpable real, en vez de 46 s de silencio.

Gracias — ¿preguntas?